Unit I :: Document Structuring and Formatting in LaTeX

SEC-03-PHY-91-R-03

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit I: Document Structuring and Formatting in LaTeX

Overview of markup-based vs. WYSIWYG systems, advantages of separating content from presentation; structure of a LaTeX document: preamble and body; plain text: whitespaces, newline, hyphens and dashes, quotation marks, special characters; rich text formatting: italics, boldface, underline and emphasis; creating lists: itemized lists, enumerated lists, nested lists; comments; labels and referencing.

1 An overview of document preparation systems

A *document preparation system* is any software that is used to *create*, *edit*, *format* and *structure* text for printing or digital viewing. Rather than just acting as digital typewriters, these systems manage the entire lifecycle of a document.

There are four main functions of a document preparation system:

a) *Content ingestion*, which means providing an interface via which the user can input and edit text.

b) *Structural tagging*, which refers to defining what the text is, e.g. chapter title, section title, footnote or a caption.

c) *Stylistic rendering*, which indicates the application of typefaces, margins, line spacing and alignment of structural elements.

d) *Output compilation*, which signifies the conversion of the working file into a printed sheet, or a ubiquitous format like PDF, HTML or EPUB.

1.1 Categories of document preparation systems

Document preparation systems can be broadly classified into two categories:

1. **Word processing software:** Such software are designed for general-purpose writing, editing, and basic layout of everyday documents like letters, reports and memos.
    - Typically, the interface is WYSIWYG which stands for *What You See Is What You Get*. This means that text and formatting appear on the screen as they will approximately look when printed.
    - Elements like images and tables are usually anchored directly to the text flow and move down the page as you type or add content.
    - The formatting approach relies heavily on direct manual formatting (such as changing font sizes, italicising or spacing on the fly) or applying predefined template styles.
    - Some examples are Microsoft Word, Google Docs and LibreOffice Writer.
2. **Typesetting software:** Such software are designed for publication-grade document composition, strict typographic control (such as kerning, hyphenation, and line-breaking algorithms) and complex scientific or multi-column layouts.
    - The interface is often markup-based. Thie means that writing plain text mixed with structural code or commands that get compiled into a final PDF, rather than a direct visual editor.
    - Automated algorithms are used to manage images and tables, page breaks, mathematical formulae and bibliographic references without manual nudging.
    - The formatting approach strictly separates content from presentation, i.e the user focuses on the content and its semantics (e.g., heading, theorem, citation etc.), while the underlying algorithm applies exact, consistent formatting rules.
    - Some examples are LaTeX, Typst and Troff.

1.2 Separation of content and formatting in scientific documentation

In scientific documentation, enabling the separation of content (e.g. text, data, structure) and formatting (e.g. font, margins, layout) allows the author to shift one's focus from presentation to substance. Such an approach is provided by LaTeX, with the following advantages:

a) Authors write in plain text using semantic tags to define structural elements, while the software handles the visual rendering in the background. Consequently, the author can spend their time refining arguments, data accuracy and prose, rather than struggling with erratic margins, font consistency or manual line breaks.

b) Global style files control the entire document's appearance. So any formatting changes instantly updates the entire manuscript consistently, thereby minimising human error.

c) Scientific documents rely heavily on interconnected data. When content is decoupled from style, the system can dynamically manage complex modules without manual intervention.

   i. Mathematical equations are rendered using precise algorithmic typesetting that maintain correct spacing and symbol scales regardless of surrounding text adjustments.

   ii. Citations, figures, tables and equations are linked to logical labels. If a figure is moved or added, the system automatically renumbers every reference throughout the document.

   iii. Bibliographies are generated directly from a central database file. Thereby the system parses the database and formats the citations perfectly according to the required style guide.

d) Submitting a paper to a different journal often requires an entirely new layout. With a separated system, authors do not need to manually reformat headers, margins or citation styles. They simply swap out the publisher's template file or package declaration, and the document automatically reshapes itself to meet the new submission guidelines.

e) Plain-text content files are exceptionally lightweight and compatible with standard version control systems like Git. Unlike proprietary binary files such as DOCX, plain-text documents allow co-authors to view precise line-by-line changes, merge collaborative edits seamlessly and track the manuscript's evolution without the risk of file corruption.

1.3 What is LaTeX?

LaTeX is free, open source, markup-based software for typesetting documents. In other words, it is a document preparation system. LaTeX was initially developed in the 1980's by Leslie Lamport, based on Donald Knuth's TeX typesetting engine.

1.3.1 Benefits of LaTeX

LaTeX is best suited for scientific and technical documents, because of its superior typesetting of mathematical formulae, consistency of high-quality output, and ease of handling large and complex documents. Some of the benefits of LaTeX are listed below:

1. Ability for automatic numbering of equations, tables, figures and sections.

2. Capabilities for cross-referencing labels.
3. Automatic generation of lists of contents, figures, tables, indexes, glossaries and bibliography.
4. Versatility of document types that can be prepared, such as articles, reports, letters, books and presentation slides.
5. Flexibility in terms of availability of various templates for a given type of document.
6. Portability in terms of source files being compatible across systems and output being device-independent.
7. Scalability in terms of handling documents of any length without performance degradation.
8. Support from a massive online community of other LaTeX users from all over the world.

1.3.2 Working with LaTeX

There are two ways of working with LaTeX:

A. Installing the LaTeX compiler and an optional IDE on one's own computer.
   - Examples of LaTeX compilers are MikTeX and TeXLive.
   - Examples of IDEs are TeXstudio and TeXmaker.
   - *Note*: Even though installing LaTeX is recommended, it may take up about 8 GB space on the hard drive and at least 2 hours for the installation.

B. Using LaTeX online *in the cloud*, with the minimum requirement being a browser on an internet-connected device.
   - Examples of online LaTeX services are Overleaf and Inscrive.
   - *Note*: Even though Overleaf is free to use, it has an upper limit on the compilation time, and this limit can be raised with a paid subscription. So for a free account, with an increase in the complexity of the document, the chances of the document not compiling also increases.

1.4 Working with LaTeX on Overleaf

Overleaf is an excellent starting point for someone who is getting started with using LaTeX without getting overwhelmed with the hassle of installation.

Some of the benefits of Overleaf are as follows:

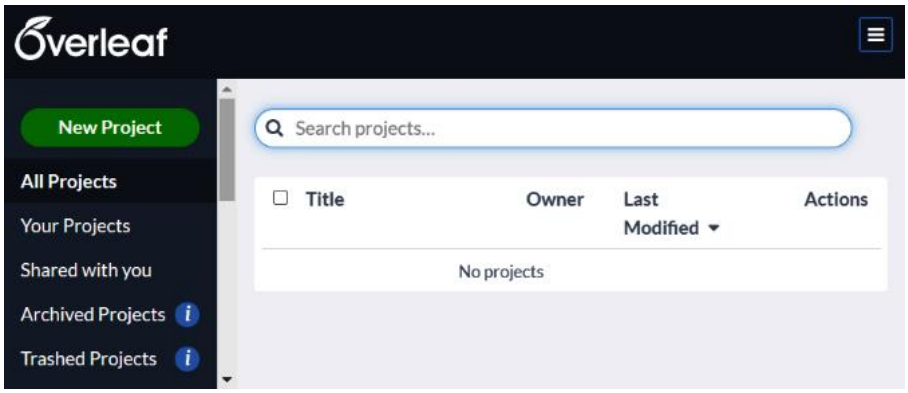
- It can be accessed on any device, such as PC, laptop, tablet or smartphone.
- It can be accessed from a locked-in work computer, where one is not allowed to install any software.

- One can access one's files on any device simply by logging in with one's password.
- One can collaborate with others to work on the same project.
- It provides a real-time view of the PDF output.
- It allows one to annotate the LaTeX code with comments and replies.

1.4.1 Creating a LaTeX project online

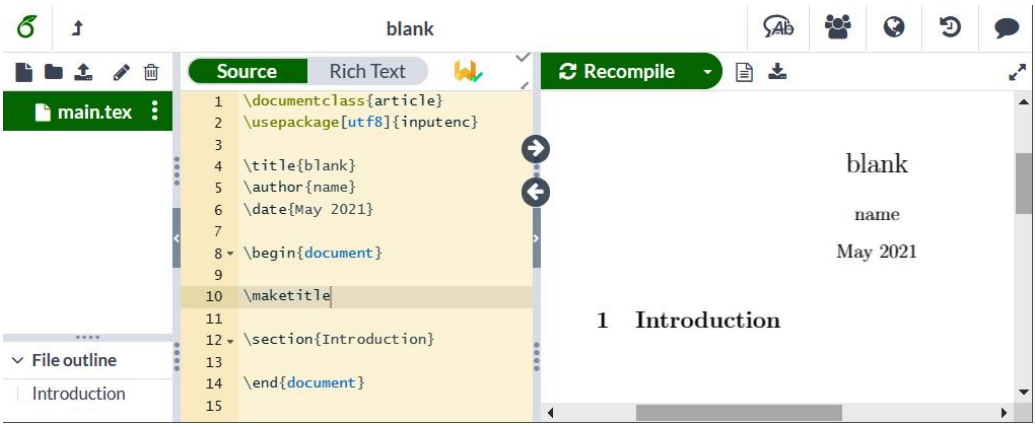
I. For a one-time registration, either click on `Register` on the Overleaf home page or visit `https://www.overleaf.com/register`. Enter an email address and choose a password.

II. To log in to Overleaf, either click on `Login` on the Overleaf home page or visit `https://www.overleaf.com/login`.

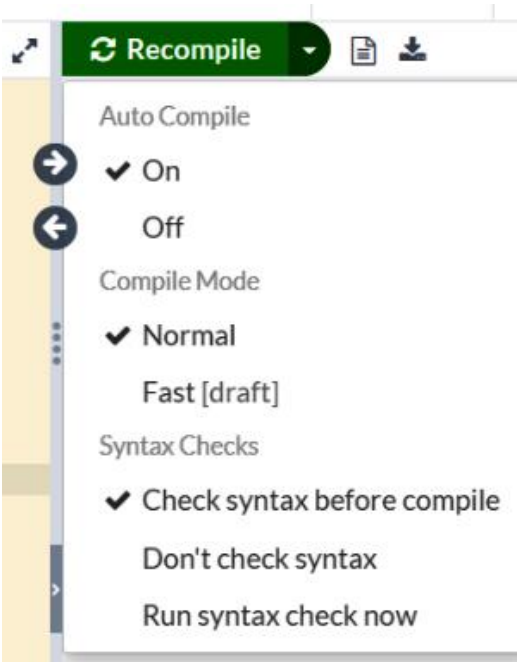


III. To create a new project, click the `New Project` button. A drop-down list will appear, where one can choose to have a blank project or one based on a template. For now, we just choose `Blank Project`.

IV. We enter a name for the project and Overleaf creates one with the bare minimum of code.



V. When any new changes are made to the project, we can view their effect on the output PDF by clicking on the `Recompile` button or pressing `Ctrl+S`. Enabling the `Auto Compile` option in the drop-down menu will ensure that the document recompiles automatically while one is typing.



2 Working with a LaTeX document

2.1 Building blocks of a LaTeX document

In LaTeX, *macros* (also called *commands*) and *environments* are the two fundamental building blocks used to define the structure, styling and behaviour of a document.

2.1.1 Macros in LaTeX

A *macro* is a single instruction given to the LaTeX compiler to perform a specific action, insert content or change a setting. All macros begin with a backslash (\), followed by a descriptive name consisting of letters or a single special character, for example `\section`.

Macros can accept *arguments* to control their behaviour. There are two types of arguments that can be passed to a macro:

1. *Mandatory arguments* are enclosed in braces { }. The macro will fail or behave incorrectly if these are omitted.
2. *Optional arguments* are enclosed in brackets [ ]. If omitted, LaTeX uses a default value.

In general, we have the following three variations in the syntax of macros:

```
\macro
\macro{mandatory arguments}
\macro[optional arguments]{mandatory arguments}
```

where the mandatory and optional arguments are separated by commas. For example, in the macro `\documentclass[12pt,a4paper]{article}`, `article` is a mandatory argument while `12pt` and `a4paper` are optional arguments.

2.1.2 Environments in LaTeX

An *environment* is a delimited block of code that applies specific structural rules, formatting, or layout settings to a specific region of content. Every environment starts with `\begin{name}` macro and ends with a matching `\end{name}` macro. Both macros require the exact same environment name as their argument.

Environments establish a *local scope*. Any formatting changes, counters or local declarations activated within an environment are automatically cancelled when the `\end{name}` macro is reached.

Environments can also accept mandatory { } and optional [ ] arguments immediately following the `\begin{name}` macro.

In general, we have the following syntax for an environment:

```
\begin{environment}[optional arguments]{mandatory arguments}
  <Content affected by the environment>
\end{environment}
```

where again the mandatory and optional arguments are separated by commas. For example, in the following environment which creates a table

```
\begin{tabular}[t]{ccc}
  abc & xy & def \\
  xyz & mn & pqr
\end{tabular}
```

`ccc` is a mandatory argument that creates three centre-aligned columns and `t` is an optional argument that specifies the vertical alignment of the table.

2.2 Structure of a LaTeX document

The simplest LaTeX document must contain the bare minimum code, as shown below. The text `Hello, world!` can be replaced with any other text.

```
\documentclass{article}
\title{My First Document}
\author{John Doe}
\date{26 Oct. 2026}

\begin{document}
  \maketitle

  \section{Introduction}
    Here is my first attempt in creating a document using \LaTeX.
\end{document}
```

2.2.1 The preamble

Every LaTeX document is an instance of a template known as a *class*, which provides customizable features. There are classes for books, journal articles, letters, presentations, posters, and many more. With the macro in the first line `\documentclass{article}`, we have chosen the `article` class, which provides a standard LaTeX template for smaller document.

We can also specify document properties using macros such as `\title`, `\author`, `\date` and any other general document-wide definitions.

This part of the document is known as the **preamble**.

```
\documentclass{article}
\title{My First Document}
\author{John Doe}
\date{26 Oct. 2026}
```

2.2.2 The document body

The \begin{document} macro demarcates the end of the preamble and the beginning of the actual document, while the \end{document} macro marks its end. Everything within this document environment appears in the document output. Here we create a title page using the \maketitle macro, a section titled Introduction using the \section macro and some text.

This part of the document contained within the document environment is known as the **document body**. Everything that follows the document body is ignored by LaTeX.

```
\begin{document}
  \maketitle

  \section{Introduction}
    Here is my first attempt in creating a document using \LaTeX.
\end{document}
```

3 Plain text formatting

In digital document processing, *plain text* refers to a raw, unformatted sequence of human-readable characters (such as standard ASCII or Unicode) that contains no embedded visual styling, font specifications, or proprietary binary metadata. When we format plain text in LaTeX we write our narrative inside a plain-text file and embed logical markup commands directly alongside the content.

3.1 Whitespaces and newlines

In LaTeX, whitespace characters (namely spaces and tabs) and newline characters (line breaks or 'enter') in the plain-text source code are interpreted according to predefined typographic rules rather than appearing literally on the output page.

3.1.1 Handling of whitespaces

- A single space typed in the source code renders as a standard inter-word space in the output document.
- Multiple consecutive spaces or tabs are automatically collapsed and treated as a single space by the compiler.
- Following a macro without any arguments(such as \LaTeX or \TeX), any trailing spaces in the source code are ignored by default.
- To force a space after such a macro, an empty pair of braces (e.g. \LaTeX{}) or an explicit control space (e.g. \LaTeX\ ) can be used.

For instance, the following LaTeX code snippet:

```
A single space typed between words in the source code renders as a standard
inter-word space.

Multiple          consecutive            spaces and         tabs are
automatically collapsed into a single space.

Trailing space after a macro without arguments such as \LaTeX is ignored.

Using empty braces such as \LaTeX{} or a control space such as \LaTeX\ forces a
 space after the macro.
```

produces the following output:

A single space typed between words in the source code renders as a standard inter-word space.
Multiple consecutive spaces and tabs are automatically collapsed into a single space.
Trailing space after a macro without arguments such as LaTeXis ignored.
Using empty braces such as LaTeX or a control space such as LaTeX forces a space after the macro.

3.1.2 Handling of newlines

- A single newline (or line break) in the plain text is treated merely as an inter-word space. Consequently, breaking a sentence across multiple lines in the source editor does not force a line break in the compiled PDF.
- One or more empty line (i.e. two or more consecutive newlines) in the plain text denotes a paragraph break.
- Alternatively, the \par macro can be used, which is functionally equivalent to an empty line.
- Upon encountering a paragraph break, LaTeX applies default paragraph formatting, such as first-line paragraph indentation (\parindent) or vertical spacing between paragraphs (\parskip).
- To force a line break without starting a new paragraph, macros such as \\ or \newline can be used. Because these macros terminate a line within the same logical paragraph, the subsequent line is not indented as a new paragraph.

For instance, the following LaTeX code snippet:

```
This sentence is split
across multiple lines in the source editor,
yet it renders as a single continuous line in the PDF.

This is the first paragraph.

The second paragraph is separated from the first by an empty line.

This paragraph is terminated explicitly using a macro.\par
This starts a new paragraph immediately following the macro.

This line is broken\\ using a manual line break in the same paragraph.

Alternatively, using the newline macro\newline forces a line break within the
same paragraph as well.
```

produces the following output:

This sentence is split across multiple lines in the source editor, yet it renders as a single continuous line in the PDF.
This is the first paragraph.
The second paragraph is separated from the first by an empty line.
This paragraph is terminated explicitly using a macro.
This starts a new paragraph immediately following the macro.
This line is broken
using a manual line break in the same paragraph.
Alternatively, using the newline macro
forces a line break within the same paragraph as well.

3.2 Hyphens and dashes

In LaTeX, horizontal lines of varying lengths are treated distinctly according to their grammatical function. Rather than relying on a single hyphen character for all purposes, LaTeX distinguishes between three primary horizontal marks based on source syntax and context.

1. *The hyphen* -
   This is a single dash - typed in standard text mode, used for joining compound words (e.g. 'well-known', 'so-called').
2. *The en-dash* –
   This is composed of two consecutive dashes -- typed in standard text mode or the macro \textendash, used primarily to indicate numerical, page or time ranges (e.g. 'pages 10–20', '1990–2000') and to link equal partners or names in concepts (e.g. 'Boyle–Mariotte law').
3. *The em-dash* —
   This is composed of three consecutive dashes --- typed in standard text mode or the macro \textemdash, used to mark an abrupt pause, a shift in thought or a parenthetical remark within a sentence (e.g. 'LaTeX—a typesetting system—is powerful').

For instance, the following LaTeX code snippet:

```
The hyphen is used for compound words, such as a well-known author, a so-called
 expert.

The en-dash is used for ranges and linking concepts/names pages 10--20 or pages
 30\textendash 50, the years 1990--2000 or 1850\textendash 1890, the Boyle--
Mariotte law or Bose\textendash Einstein condensate.

The em-dash is used for parenthetical remarks or shifts in thought \LaTeX---a
typesetting system---is powerful or Python\textemdash a high-level language\
textemdash is versatile.
```

produces the following output:

The hyphen is used for compound words, such as a well-known author, a so-called expert.
The en-dash is used for ranges and linking concepts/names pages 10–20 or pages 30–50, the years 1990–2000 or 1850–1890, the Boyle–Mariotte law or Bose–Einstein condensate.
The em-dash is used for parenthetical remarks or shifts in thought LaTeX—a typesetting system—is powerful or Python—a high-level language—is versatile.

3.3 Quotation marks

In LaTeX, quotation marks require specific handling to ensure proper typographical opening and closing glyphs. Standard typewriter double quotes (") should not be used in standard text flow, as they produce straight quotes rather than proper curly quotation marks. LaTeX uses grave accents (backticks) for opening quotes and apostrophes for closing quotes as follows:

- Single opening quote is a single grave accent or backtick (`).

- Single closing quote is a single apostrophe or right quote (').
- Double opening quotes are two consecutive grave accents or backticks (``).
- Double closing quotes are two consecutive apostrophes or right quotes ('').

For instance, the following LaTeX code snippet:

```
‘This is a single-quoted phrase’

‘‘This is a double-quoted phrase’’

‘‘She said, ‘Hello world!’ before leaving.’’
```

produces the following output:

‘This is a single-quoted phrase’
“This is a double-quoted phrase”
“She said, ‘Hello world!’ before leaving.”

3.4 Special reserved characters

In LaTeX, plain-text input is interpreted through a set of syntax rules where certain symbols serve as control characters rather than literal printable text. Ten specific characters are reserved by the TeX engine for formatting, command execution, and structural scoping.

3.4.1 Reserved characters

Each of the following reserved characters carries a dedicated syntactic role within LaTeX source code:

1. Backslash (\) starts a LaTeX macro or environment declaration.
2. Braces ({ and }) enclose mandatory command arguments, group text or delimit local scopes.
3. Percentage sign (%) introduces a single-line comment; LaTeX ignores everything following the % symbol until the end of the line.
4. Dollar sign ($) marks the beginning and end of inline math mode.
5. Ampersand (&) acts as an alignment or column separator within tabular material, matrices and math alignment environments.
6. Hash sign (#) references macro parameter positions (such as #1 or #2) when defining new commands.
7. Underscore (_) specifies subscripts in mathematical expressions.
8. Caret (^) specifies superscripts in mathematical expressions.
9. Tilde (~) inserts a non-breaking (unbreakable) inter-word space that prevents line breaks between words.

3.4.2 Escaping reserved characters

To print these reserved characters as literal text in the output, seven of them can be printed simply by placing a preceding backslash (\) before the character, as shown below:

```
\%    \$    \&    \#    \_    \{    \}
```

which produces the following output:

% $ & # _ { }

4 Rich text formatting

In LaTeX, *rich text formatting* modifies font shape, weight, and decoration to establish visual hierarchy and highlight specific text elements. Rather than applying arbitrary physical changes directly, LaTeX distinguishes between physical font styling (direct typeface modifications) and semantic/logical emphasis.

4.1 Typeface modification syntax

Typeface-changing mechanisms in LaTeX generally come in two distinct syntactical forms:

1. *Argument-based font commands*: This formatting applies strictly to the text inside the curly braces { }. Once the closing brace is reached, LaTeX automatically resumes the previous text formatting.

   The general syntax is: `\textXX{argument}`
   where `XX` could be replaced with `it` for *italics*, `bf` for **boldface** and `tt` for `teletype`.

   For instance, the following LaTeX code snippet:

   ```
   A chunk of text \textit{can be italicized}, or it can be \textbf{displayed
   in boldface}, or it can be \texttt{converted into typewriter teletype}
   using argument-based font commands.
   ```

   produces the following output:

   A chunk of text *can be italicized*, or it can be **displayed in boldface**, or it can be `converted into typewriter teletype` using argument-based font commands.

2. *State-switching font declarations*: Such declarations take no arguments. Instead, they act as switches that alter the formatting of all subsequent text. To limit their scope, declarations must be enclosed within a pair of curly braces { } or an environment to form a local group.

   The general syntax is: `\XXshape` or `\XXseries`

   For instance, the following LaTeX code snippet:

   ```
   The effect of declaration switches is {\bfseries confined within local
   brace groups}. Outside the brace group, the font state is restored.

   We can switch {\itshape to a different font state} using a new local brace
   group.
   ```

   produces the following output:

   The effect of declaration switches is **confined within local brace groups**. Outside the brace group, the font state is restored.
   We can switch *to a different font state* using a new local brace group.

4.2 Font families

A *typeface* is a visual design for letters, such as Computer Modern or Times. In LaTeX, a typeface is organised as a *font family*, which comprises of the design, together with all of its variants (upright, italic, bold, small capitals, and so on). A *font* is one concrete member of a family at a specific size, for example "Computer Modern Roman, bold, 12 pt".

Every LaTeX document works with three logical families:

1. Roman family

   This is written with Roman font family.

   - Such font has small lines or *strokes*, known as *serifs*, which are attached to letters.
   - This is generally used to typeset printed documents, such as reports, articles or letters.
   - In LaTeX, this is the default font.

2. Sans serif family

   This is written with sans serif font family.

   - In such font, the strokes or serifs are missing.
   - This is generally used to typeset digital display content, such as slides and social media flyers.

3. Typewriter (or teletype) family

   `This is written with typewriter font family.`

   - In such font, all the characters are monospaced, i.e. have the same width.
   - This is generally used to typeset URLs (i.e. webpage links) and program codes.

In the table below, the font commands and declarations for the differen font families are given

Command	Declaration	Meaning
`\textrm{...}`	`\rmfamily`	roman family
`\textsf{...}`	`\sffamily`	sans-serif family
`\texttt{...}`	`\ttfamily`	`typewriter family`
`\textbf{...}`	`\bfseries`	**boldface font**
`\textmd{...}`	`\mdseries`	medium font
`\textit{...}`	`\itshape`	*italic shape*
`\textsl{...}`	`\slshape`	*slanted shape*
`\textsc{...}`	`\scshape`	SMALL CAPS SHAPE
`\textup{...}`	`\upshape`	UPRIGHT SHAPE
`\textnormal{...}`	`\normalfont`	default font

4.3 Font sizes in LaTeX

A LaTeX document has a base font size, which holds good for the entire document. The standard document classes support three base font sizes: `10pt` (default), `11pt` and `12pt`.

- For other standard base sizes, we can use the `extsizes` package.

Once a base font size is set for a document, there are 10 font size declarations which modify the font size of the text within a local scope to further modify their sizes, relative the base font size. These size declarations are demonstrated in the table below.

Declaration	Example	Effect
`\tiny`	`{\tiny Tiny text}`	Tiny text
`\scriptsize`	`{\scriptsize Script-size text}`	Script-size text
`\footnotesize`	`{\footnotesize Footnote-size text}`	Footnote-size text
`\small`	`{\small Small text}`	Small text
`\normalsize`	`{\normalsize Default text}`	Default text
`\large`	`{\large Large text}`	Large text
`\Large`	`{\Large Larger text}`	Larger text
`\LARGE`	`{\LARGE Even larger text}`	Even larger text
`\huge`	`{\huge Huge text}`	Huge text
`\Huge`	`{\Huge More huge text}`	More huge text

4.4 Underlining text

The built-in macro `\underline{...}` draws a straight horizontal line under its argument. However, this does not allow automatic line breaking or hyphenation across long passages.

For enhanced underlining, we can use the `ulem` package which allows multi-line underlined text with automatic hyphenation. To do so, first we need to include the following line in the preamble (anywhere below the `\documentclass` macro):

```
\usepackage[normalem]{ulem}
```

following which we can use different macros that provide us with many options for underlines and other decorations. Some of the popular macros are given in the following table.

Macro	Example	Effect
`\uline{...}`	`\uline{Underlined text}`	Underlined text
`\uuline{...}`	`\uuline{Double underlined text}`	Double underlined text
`\uwave{...}`	`\uwave{Wavy underlined text}`	Wavy underlined text
`\sout{...}`	`\sout{Struck-out text}`	~~Struck-out text~~
`\xout{...}`	`\xout{Crossed-out text}`	Crossed-out text

4.5 Text emphasis

To draw attention to a word or phrase, we can use the `\emph{...}` macro. This is a *logical markup* which says that the text within { } is emphasized and leaves it to LaTeX to decide how it does so. For instance, the following

```
This is \emph{very} important!
```

produces the output

This is *very* important!

Difference between emphasis and italics: The `\textit` macro always selects the italic shape, regardless of the surroundings, while the `\emph` macro toggles relative to the current shape:

- inside upright text it switches to italic
- inside italic text it switches back to upright

This keeps the emphasis visible when it is nested or placed in an italic context, such as the body of a theorem.

For instance, the following LaTeX code snippet:

```
Inside upright text, using the emphasis macro \emph{changes the text to italics
}, while the rest remains upright.

{\itshape Inside italic text, using the emphasis macro \emph{changes the text
to upright}, while the rest remains italicized.}
```

produces the following output:

Inside upright text, using the emphasis macro *changes the text to italics*, while the rest remains upright.
Inside italic text, using the emphasis macro changes the text to upright, *while the rest remains italicized.*

5 Creating lists

Lists in LaTeX provide a structured layout to present key points, sequential steps or categorized information. In LaTeX, all lists are constructed as environments, with each entry marked by the `\item` macro.

5.1 Itemized lists

An *itemized list* (or un-numbered list) is used when the order of items is flexible or unimportant. The general syntax for creating an itemized list is:

```
\begin{itemize}
  \item First item
  \item Second item
  \item Third item
\end{itemize}
```

The above code produces the following output:

- First item
- Second item
- Third item

The default symbol for any entry is a bullet point. However, it can be changed by inserting another symbol as an option to the `\item` macro. Some examples are as follows:

```
\begin{itemize}
  \item[-] First item
  \item[--] Second item
  \item[$\circ$] Third item
  \item[$\diamond$] Fourth item
  \item[$\star$] Fifth item
\end{itemize}
```

which produces the output

- First item
– Second item
∘ Third item
◇ Fourth item
⋆ Fifth item

5.2 Enumerated lists

An *enumerated list* (or numbered list) is used when the sequential order of items is essential, such as in step-by-step procedures or rankings. The general syntax for creating an enumerated list is:

```
\begin{enumerate}
  \item First item
  \item Second item
  \item Third item
\end{enumerate}
```

The above code produces the following output:

1. First item
2. Second item
3. Third item

By default, arabic numerals are displayed for any entry. However, it can be changed by including the `enumerate` package in the preamble:

```
\usepackage{enumerate}
```

Following this, the numeral and numbering format can be mentioned as an optional argument to the `enumerate` environment. For example, to typeset the numbering using a capital letter from the alphabet followed by a dot, we use the following:

```
\begin{enumerate}[A.]
  \item First item
  \item Second item
\end{enumerate}
```

which produces the output

A. First item

B. Second item

Again, to typeset the numbering using roman numerals in small letters enclosed in parentheses, we use the following:

```
\begin{enumerate}[(i)]
  \item First item
  \item Second item
  \item Third item
\end{enumerate}
```

which produces the output

i) First item

ii) Second item

iii) Third item

5.3 Nested lists

Lists can be *nested* by placing a child list environment directly beneath an `\item` of a parent list. It is possible to nest homogeneous lists (e.g. `itemize` inside `itemize`, or `enumerate` inside `enumerate`) or heterogeneous lists (e.g. `itemize` inside `enumerate` or vice versa).

The following code demonstrates some nested lists:

```
\begin{itemize}
  \item This is an item.
  \begin{itemize}
    \item This is a sub-item.
    \item This is another sub-item.
  \end{itemize}
  \item This is another item.
\end{itemize}

\begin{enumerate}
  \item This is the first item.
  \item This is the second item.
  \begin{enumerate}
    \item This is the first sub-item.
    \item This is the second sub-item.
  \end{enumerate}
\end{enumerate}

\begin{itemize}
  \item This is an item.
  \begin{enumerate}
    \item This is the first sub-item.
    \item This is the second sub-item.
  \end{enumerate}
  \item This is another item.
\end{itemize}

\begin{enumerate}
  \item This is the first item.
  \item This is the second item.
  \begin{itemize}
    \item This is a sub-item.
    \item This is another sub-item.
  \end{itemize}
\end{enumerate}
```

which produces the following output:

- This is an item.
  - This is a sub-item.
  - This is another sub-item.
- This is another item.

1. This is the first item.
2. This is the second item.
   (a) This is the first sub-item.
   (b) This is the second sub-item.

- This is an item.
  1. This is the first sub-item.
  2. This is the second sub-item.
- This is another item.

1. This is the first item.
2. This is the second item.
   - This is a sub-item.
   - This is another sub-item.

6 Using comments in LaTeX

In LaTeX, *comments* allow authors to insert non-printing notes, suppress undesired whitespace, or temporarily disable sections of code during document development. Comments are ignored by the LaTeX compiler and do not appear in the generated output document.

A comment is introduced using the percent character (%). Upon encountering a % symbol in standard text mode, LaTeX ignores the % itself and everything following it on that same line until the end-of-line character.

For instance, the following LaTeX code snippet:

```
This is a regular text. % This comment is ignored by LaTeX

A comment can be used %to ignore text till the end of the line.

% An entire line can also be turned into a comment.
```

produces the following output:

This is a regular text.
A comment can be used

6.1 Utility of comments

As in any programming language, comments in LaTeX have numerous utilities, some of which are outlined below.

1. Authors can leave descriptive annotations, structural markers, or to-do notes for themselves or co-authors without affecting the typeset document. LaTeX templates frequently use comments to guide users on required preamble setups or placeholder entries.
2. Rather than deleting commands or text paragraphs when testing alternative layouts, authors can “comment out” lines by prefixing them with %. This preserves the original code, allowing it to be easily restored by simply removing the percent sign.
3. Because a single line break in source code acts as a space character, placing a % at the very end of a line (immediately following the last character without an intervening space) masks the newline. This technique is widely used when constructing macros, defining custom environments, or placing inline elements side-by-side where extra whitespace would disrupt alignment or page layout.

7 Labels and cross-referencing

In LaTeX, cross-referencing allows authors to refer dynamically to numbered document elements (such as chapters, sections, figures, tables, equations, footnotes and list items) without manually hardcoding numbers. When structural elements are inserted, deleted or reordered, LaTeX automatically re-evaluates all counter positions and updates references across the document.

7.1 Macros for cross-referencing

Cross-referencing in standard LaTeX relies on three fundamental commands:

1. `\label{key}`: This assigns a unique symbolic identifier (or `key`) to the currently active structural element or counter at that exact position in the source text.
2. `\ref{key}`: This retrieves and prints the logical sequence number associated with `key`.
3. `\pageref{key}`: This retrieves and prints the page number on which the corresponding `\label{key}` command is located.

7.2 Placement rules and conventions

- The `\label` macro must be placed immediately after (or within) the command or environment that updates the active reference counter.
- In floating environments (such as `figure` or `table`), `\label` must follow the `\caption` command; placing `\label` before `\caption` incorrectly binds the key to the enclosing section counter rather than the figure or table counter.
- Although keys can be named arbitrarily, establishing consistent prefix conventions prevents duplicate keys and improves code maintainability:
  - `sec:` for chapters and sections (e.g. `sec:intro`)
  - `fig:` for figures (e.g. `fig:chart`)
  - `tab:` for tables (e.g. `tab:data`)
  - `eq:` for mathematical equations (e.g., `eq:einstein`)
  - `item:` for enumerated list entries (e.g. `item:step1`)
  - `fn:` for footnotes (e.g. `fn:note1`)